Algoritmos de Escalonamento de Processos em Sistemas Operacionais

Autor: Jordan Santos Ramos – FSG – jordan.ramos@cs.edu.br

Resumo

O algoritmo de escalonamento de processos é uma das principais funções atribuídas a um sistema operacional, onde ele deve decidir, em um determinado tempo, a ordem, duração e qual processo deve ser executado na CPU. Este artigo reúne, em um contexto mais amplo, as obras mais relevantes a este tema, abrangendo a discussão desde as mais antigas propostas, tais como FIFO e Round-Robin, até as mais contemporâneas, como o Completely Fair Scheduler (CFS) do Linux e o EEVDF. Tratando das características de cada modelo. A pesquisa evidencia a crescente necessidade de algoritmos de escalonamento, conforme a evolução dos ambientes computacionais, e algumas distinções entre seus paradigmas e comportamentos.

Palavras-chave: Escalonamento de Processos; Sistemas Operacionais; Gerenciamento de CPU; Algoritmos de Escalonamento; Desempenho de Sistemas

1. Introdução

O escalonamento de processos é uma das funções mais críticas de um sistema operacional moderno. Dentro de um sistema computacional onde vários processos estão concorrendo, o escalonador é o juiz que determina qual processo deve executar, por quanto tempo e em qual momento.

A eficiência do escalonamento determina a satisfação do usuário, o throughput e a utilização dos recursos do sistema. Um algoritmo de escalonamento que seja mal projetado fará com que o sistema tenha um desempenho muito abaixo do desejado, com um tempo de resposta intolerável e lentidão. Por outro lado, uma boa implementação projetada com cuidado poderá levar uma grande produtividade aliada a uma grande satisfação dos usuários.

Este trabalho pretende elaborar uma revisão bibliográfica sobre o escalonamento de processos, abordando desde as abordagens mais clássicas das décadas de 60 e 70 até as práticas atuais em sistemas operacionais como o Linux e o Windows. Serão explorados os conceitos fundamentais e as características técnicas, vantagens e desvantagens, e limitações de cada proposta.

2. Metodologia

A pesquisa foi realizada nas principais bases de dados acadêmicas e repositórios de conhecimento técnico.

2.1 Bases de Dados Consultadas

IEEE Xplore Digital Library
ACM Digital Library
Google Scholar
Website oficial do kernel Linux

2.2 Critérios de Seleção

Foram selecionados trabalhos publicados entre 1960 e 2024, livros-texto de referência na área e documentação técnica oficial. Os critérios de inclusão foram: relevância para algoritmos de escalonamento, e reputação no campo.

3 Revisão da Literatura

3.1 Fundamentos do Escalonamento

O escalonamento de processos é fundamentado em objetivos específicos que variam conforme o tipo de sistema. Segundo Tanenbaum e Bos (2016), os principais objetivos incluem: maximizar a utilização da CPU, garantir justiça entre processos, minimizar o tempo de resposta para aplicações interativas e maximizar o throughput em sistemas batch.

O escalonamento pode ser não-preemptivo, onde o processo executa até terminar ou bloquear voluntariamente, ou preemptivo, onde o sistema operacional pode interromper a execução de um processo para dar lugar a outro. A escolha entre estas abordagens depende dos requisitos do sistema e das garantias de tempo real necessárias.

3.2 Algoritmos Clássicos

3.2.1 First-Come, First-Served (FCFS)

O algoritmo FCFS é o mais simples de todos os algoritmos de escalonamento. Os processos são executados na ordem de chegada, sem preempção. Embora seja fácil de implementar e compreender, sofre do problema conhecido como "convoy effect", onde processos curtos devem esperar por processos longos que chegaram antes (Silberschatz et al., 2018).

3.2.2 Shortest Job First (SJF)

O SJF seleciona o processo com o menor tempo de execução estimado. Teoricamente, minimiza o tempo médio de espera, sendo ótimo nesse aspecto. No entanto, apresenta duas limitações significativas: a dificuldade de prever o tempo de execução e o risco de starvation para processos longos (Stallings, 2015).

```
Pseudocódigo:

enquanto processos não vazio:
    p = processos.processo com menor tempo_execução()
    executar(p)
    remover p de processos
```

3.2.3 Round-Robin (RR)

O Round-Robin é provavelmente o algoritmo mais utilizado em sistemas interativos. Cada processo recebe um quantum de tempo (tipicamente 10-100ms) para execução. Quando o quantum expira, o processo é movido para o final da fila. Este algoritmo garante que nenhum processo monopolize a CPU e proporciona tempos de resposta previsíveis (Tanenbaum & Bos, 2016).

Quantum no Round-Robin:

A escolha do quantum é crítica: um quantum muito pequeno gera overhead excessivo de chaveamento de contexto, enquanto um quantum muito grande aproxima o comportamento ao FCFS. Valores típicos variam entre 20-50ms em sistemas modernos.

3.3 Algoritmos Modernos

3.3.1 Completely Fair Scheduler (CFS) - Linux

O CFS, implementado por Ingo Molnar e integrado ao kernel Linux 2.6.23, representa um marco na evolução dos algoritmos de escalonamento. Diferentemente dos algoritmos clássicos baseados em filas, o CFS utiliza uma árvore rubro-negra ordenada por virtual runtime (vruntime).

O vruntime representa o tempo de CPU que um processo recebeu, ajustado por sua prioridade. O CFS sempre seleciona o processo com menor vruntime (nó mais à esquerda da árvore), garantindo justiça perfeita na distribuição de CPU com granularidade de nanossegundos.
O CFS não utiliza *timeslices* fixos. Em vez disso, trabalha com o conceito de *target latency* (latência alvo), tipicamente 20ms, que representa o tempo mínimo para que todos os processos executáveis recebam ao menos uma fatia de CPU (Love, 2010).

3.3.2 EEVDF (Earliest Eligible Virtual Deadline First)

A partir do kernel Linux 6.6 (2024), o sistema começou a transição do CFS para o EEVDF, proposto por Peter Zijlstra. O EEVDF introduz o conceito de **lag** (atraso) para determinar se um processo recebeu sua parte justa de CPU. Processos com *lag* positivo (devem receber CPU) são elegíveis, e entre eles, o processo com o deadline virtual mais próximo é selecionado.

3.3.3 Escalonador do Windows

O Windows utiliza um sistema de prioridades multinível com 32 níveis (0-31), divididos em tempo real (16-31) e dinâmicos (0-15). O escalonador implementa filas de feedback multinível, onde processos podem ter suas prioridades ajustadas dinamicamente. Além disso, o Windows oferece "foreground boost", alocando mais recursos para aplicações em primeiro plano (Russinovich et al., 2017).

3.4 Tabela Comparativa

Algoritmo	Tipo	Vantagens	Desvantagens
FIFO/FCFS	Não-preemptivo	Simples, justo por ordem de chegada	Convoy effect, tempo médio de espera elevado
SJF	Não-preemptivo	Minimiza tempo médio de espera	Overhead de chaveamento, starvation
SRTF	Preemptivo	Tempo de resposta ótimo	Overhead de chaveamento, starvation
ROUND-ROBIN	Preemptivo	Justo, tempo de resposta previsível	Desempenho dependente do quantum
Prioridades	Preemptivo	Flexível, atende diferentes necessidades	Starvation sem aging
CFS (Linux)	Preemptivo	Justiça perfeita, granularidade nanossegundos	Complexidade de implementação

3.5 Escalonamento em Arquiteturas Heterogêneas

Com o advento de processadores heterogêneos (P-cores + E-cores), como os Intel de 12ª geração, surgiu a necessidade de escalonadores conscientes da arquitetura. O Intel Thread Director é um microcontrolador integrado que fornece informações ao escalonador do Windows 11 sobre qual núcleo é mais adequado para cada thread, considerando características de desempenho e eficiência energética.

4. Discussão e Análise Crítica

4.1 Evolução Histórica

A evolução dos algoritmos de escalonamento reflete as mudanças nos requisitos computacionais. Nos primórdios (décadas de 1960-70), sistemas batch priorizavam *throughput*, favorecendo algoritmos como FCFS e SJF. Com a popularização de sistemas time-sharing nos anos 80, Round-Robin tornou-se predominante. A era moderna (2000+) demanda justiça perfeita e baixa latência, resultando em algoritmos sofisticados como CFS e EEVDF.

4.2 Desafios Atuais

Os sistemas operacionais contemporâneos enfrentam desafios únicos:

- Heterogeneidade: Processadores com núcleos de diferentes capacidades requerem decisões inteligentes de *placement*
- Eficiência Energética: Balanceamento entre desempenho e consumo de energia, especialmente em dispositivos móveis
- Virtualização: Escalonamento de múltiplas *VMs* com requisitos distintos no mesmo hardware
- Tempo Real: Garantias determinísticas em sistemas críticos sem sacrificar desempenho geral

4.3 Comparação Crítica entre Abordagens de Escalonamento

A análise entre os algoritmos mostra que cada um corresponde a objetivos distintos, refletindo as demandas históricas e arquiteturais de seus contextos. Algoritmos clássicos como FCFS, SJF e RR privilegiavam simplicidade, enquanto as soluções modernas CFS, EEVDF e o escalonador do Windows corresponde a demanda por adaptabilidade, justiça e responsividade em cenários complexos.

Os algoritmos não-preemptivos, apesar de eficientes em cargas estáveis, falham em ambientes interativos, nos quais latência é determinante. FCFS sofre com o convoy effect e SJF, embora teoricamente ótimo em espera média, torna-se impraticável devido à necessidade de estimar tempos de execução. Em contrapartida, algoritmos preemptivos como RR introduziram responsividade, mas ao custo de maior overhead de chaveamento e sensibilidade ao quantum, exigindo afinamento constante.

A transição para algoritmos baseados em justiça proporcional representa uma mudança de paradigma. O CFS eliminou filas tradicionais e adotou a noção de vruntime para distribuir CPU de forma quase contínua, aproximando-se de um modelo idealizado de *fairness* (justiça/equidade).

O Windows segue uma linha distinta, priorizando responsividade perceptível por meio de prioridades dinâmicas, com o realce de foreground e heurísticas específicas para cargas mistas. Essa abordagem, embora eficaz em ambientes desktop, tende a ser menos previsível e mais dependente de heurísticas proprietárias, o que dificulta sua avaliação formal. ~~(Empiricamente posso afirmar que é ruim)~~

Quando observados ao todo, esses algoritmos mostram um conjunto de tensões recorrentes: justiça versus desempenho agregado, simplicidade versus adaptabilidade.

5 Conclusão

Esta revisão bibliográfica apresentou uma análise dos algoritmos de escalonamento de processos, desde fundamentos históricos até implementações modernas, com cada contexto demandando abordagens específicas.

Os algoritmos clássicos (FCFS, SJF, Round-Robin, Prioridades) estabeleceram os princípios que ainda guiam o design de escalonadores modernos. O CFS do Linux revolucionou o conceito de justiça com granularidade de nanossegundos, enquanto o EEVDF promete melhorias nas qualidades desejáveis daquele contexto.

Os desafios contemporâneos, processadores heterogêneos, garantias de tempo real, responsividade, exigem soluções cada vez mais sofisticadas.

Em suma, o escalonamento de processos permanece como uma área de pesquisa ativa e sempre presente em sistemas operacionais multiprogramados, amarrada à sua evolução com a evolução destes, e é guiada pelos objetivos do sistema.

Referências

[1]
TANENBAUM, A. S.; BOS, H. Sistemas Operacionais Modernos.. 4ª ed. São Paulo: Pearson, 2016.
[2]
SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Fundamentos de Sistemas Operacionais.. 10ª ed. Rio de Janeiro: LTC, 2018.
[3]
STALLINGS, W. Operating Systems: Internals and Design Principles.. 8th ed. Boston: Pearson, 2015.
[4]
LOVE, R. Linux Kernel Development.. 3rd ed. Upper Saddle River: Addison-Wesley, 2010.
[5]
RUSSINOVICH, M. E.; SOLOMON, D. A.; IONESCU, A. Windows Internals, Part 1: System architecture, processes, threads, memory management.. 7th ed. Redmond: Microsoft Press, 2017.
[6]
MOLNAR, I. CFS Scheduler Design.. Linux Kernel Documentation, 2007.
[7]
ZIJLSTRA, P. EEVDF Scheduler.. Linux Kernel Mailing List, 2023.
[8]
INTEL CORPORATION Intel Thread Director: Optimizing Performance and Efficiency on 12th Gen Intel Core Processors.. Intel Technical Documentation, 2021.
[9]
BUTTAZZO, G. C. Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications.. 3rd ed. New York: Springer, 2011.
[10]
ANDERSON, T.; DAHLIN, M. Operating Systems: Principles and Practice.. 2nd ed. Seattle: Recursive Books, 2014.